```java
public class Q1 {
    public static void main(String[] args) {

        try {
            int a = 10 / 0;
        } catch (ArithmeticException e) {
            System.out.println("ArithmeticException: " + e.getMessage());
        }

        try {
            String s = null;
            System.out.println(s.length());
        } catch (NullPointerException e) {
            System.out.println("NullPointerException");
        }

        try {
            int[] arr = {10, 20, 30};
            System.out.println(arr[5]);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("ArrayIndexOutOfBoundsException");
        }

        try {
            int n = Integer.parseInt("abc");
        } catch (NumberFormatException e) {
            System.out.println("NumberFormatException");
        }
    }
}
```

Output

```
ArithmeticException: / by zero
NullPointerException
ArrayIndexOutOfBoundsException
NumberFormatException
```

```java
public class Q2 {
    public static void main(String[] args) {

        try {
            System.out.println("Outer try");

            try {
                int a = 10 / 0;
            } catch (NullPointerException e) {
                System.out.println("Inner catch");
            }

        } catch (ArithmeticException e) {
            System.out.println("Outer catch: ArithmeticException");
        }

        try {
            String s = null;

            try {
                System.out.println(s.length());
            } catch (ArithmeticException e) {
                System.out.println("Inner catch");
            }

        } catch (NullPointerException e) {
            System.out.println("Outer catch: NullPointerException");
        }
    }
}
```

Output

```
Outer try
Outer catch: ArithmeticException
Outer catch: NullPointerException
```

```java
import java.util.Scanner;

public class Q3 {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        try {
            System.out.print("Enter first number: ");
            double a = Double.parseDouble(sc.nextLine());

            System.out.print("Enter second number: ");
            double b = Double.parseDouble(sc.nextLine());

            System.out.print("Enter operator: ");
            String op = sc.nextLine();

            double result;

            switch (op) {
                case "+":
                    result = a + b;
                    break;
                case "-":
                    result = a - b;
                    break;
                case "*":
                    result = a * b;
                    break;
```

```
                case "/":
                    result = a / b;
                    break;
                default:
                    throw new IllegalArgumentException("Invalid operator");
            }

            System.out.println("Result: " + result);

        } catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero");
        } catch (NumberFormatException e) {
            System.out.println("Invalid numeric input");
        } catch (IllegalArgumentException e) {
            System.out.println(e.getMessage());
        }

        sc.close();
    }
}
```

Output

```
Enter first number: 5
Enter second number: 43
Enter operator: -
Result: -38.0
```

Q4

```java
import java.util.Scanner;
public class ResultProcessing {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of subjects: ");
        int n = sc.nextInt();
        int[] marks = new int[n];
        try {
            for (int i = 0; i < n; i++) {
                System.out.print("Enter marks for subject " + (i + 1) + ": ");
                int m = sc.nextInt();
                if (m < 0 || m > 100)
                    throw new InvalidMarksException("Invalid marks for subject " + (i + 1) + ": " + m);
                marks[i] = m;
            }
            int total = 0;
            for (int m : marks) total += m;
            double percentage = (double) total / n;
            char grade;
            if (percentage >= 90) grade = 'A';
            else if (percentage >= 75) grade = 'B';
            else if (percentage >= 60) grade = 'C';
            else if (percentage >= 40) grade = 'D';
            else grade = 'F';
            System.out.println("Total: " + total);
            System.out.println("Percentage: " + percentage);
            System.out.println("Grade: " + grade);
        } catch (InvalidMarksException e) {
            System.out.println("Error: " + e.getMessage());
        }
        sc.close();
    }
}
class InvalidMarksException extends Exception {
    InvalidMarksException(String msg) { super(msg); }
}
```

Output

```
Enter number of subjects: 3
Enter marks for subject 1: 90
Enter marks for subject 2: 99
Enter marks for subject 3: 97
Total: 286
Percentage: 95.33333333333333
Grade: A
```

Q5

```java
import java.util.Scanner;
public class Q5 {
    static BankAccount findAccount(BankAccount[] accounts, int number)
            throws AccountNotFoundException {
        for (BankAccount account : accounts) {
            if (account.accountNumber == number) return account;
        }
        throw new AccountNotFoundException("Account not found");
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        BankAccount[] accounts = {
            new BankAccount(101, 10000),
            new BankAccount(102, 5000)
        };

        try {
            System.out.print("Enter account number: ");
            int number = sc.nextInt();
            BankAccount account = findAccount(accounts, number);

            System.out.print("Enter deposit amount: ");
            double deposit = sc.nextDouble();
            account.deposit(deposit);
            account.displayBalance();

            System.out.print("Enter withdrawal amount: ");
            double withdrawal = sc.nextDouble();
            account.withdraw(withdrawal);
            account.displayBalance();

        } catch (AccountNotFoundException e) {
            System.out.println(e.getMessage());
        } catch (InvalidAmountException e) {
            System.out.println(e.getMessage());
        } catch (InsufficientBalanceException e) {
            System.out.println(e.getMessage());
        }
```

```
    }
}

class InsufficientBalanceException extends Exception {
    InsufficientBalanceException(String message) { super(message); }
}

class InvalidAmountException extends Exception {
    InvalidAmountException(String message) { super(message); }
}

class AccountNotFoundException extends Exception {
    AccountNotFoundException(String message) { super(message); }
}

class BankAccount {
    int accountNumber;
    double balance;

    BankAccount(int accountNumber, double balance) {
        this.accountNumber = accountNumber;
        this.balance = balance;
    }

    void deposit(double amount) throws InvalidAmountException {
        if (amount <= 0) throw new InvalidAmountException("Deposit amount must be positive");
        balance += amount;
    }

    void checkWithdrawal(double amount) throws InvalidAmountException, InsufficientBalanceException {
        if (amount <= 0) throw new InvalidAmountException("Withdrawal amount must be positive");
        if (amount > balance) throw new InsufficientBalanceException("Insufficient balance");
    }

    void withdraw(double amount) throws InvalidAmountException, InsufficientBalanceException {
        checkWithdrawal(amount);
        balance -= amount;
    }

    void displayBalance() {
        System.out.println("Account Balance: " + balance);
```

Output

```
Enter account number: 101
Enter deposit amount: 49000
Account Balance: 59000.0
Enter withdrawal amount: 6000
Account Balance: 53000.0
```

```java
import java.util.Scanner;
public class LoginSystem {
    static final String USERNAME = "admin";
    static final String PASSWORD = "pass123";
    static final int MAX_ATTEMPTS = 3;

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int attempts = 0;
        boolean locked = false;

        while (attempts < MAX_ATTEMPTS && !locked) {
            try {
                System.out.print("Enter username: ");
                String uname = sc.next();
                if (!uname.equals(USERNAME))
                    throw new InvalidUsernameException("Invalid username");

                System.out.print("Enter password: ");
                String pass = sc.next();
                if (!pass.equals(PASSWORD))
                    throw new InvalidPasswordException("Invalid password");

                System.out.println("Login successful");
                break;

            } catch (InvalidUsernameException | InvalidPasswordException e) {
                attempts++;
                System.out.println("Error: " + e.getMessage() + ". Attempts left: " + (MAX_ATTEMPTS - attempts));
                if (attempts == MAX_ATTEMPTS) {
                    locked = true;
                    try {
                        throw new AccountLockedException("Account locked due to too many failed attempts");
                    } catch (AccountLockedException ex) {
                        System.out.println("Error: " + ex.getMessage());
                    }
                }
            } finally {
                System.out.println("Attempt processed");
            }
        }
        sc.close();
    }
}

class InvalidUsernameException extends Exception {
    InvalidUsernameException(String msg) { super(msg); }
}

class InvalidPasswordException extends Exception {
    InvalidPasswordException(String msg) { super(msg); }
}

class AccountLockedException extends Exception {
    AccountLockedException(String msg) { super(msg); }
}
```

Output

```
Enter username: admin
Enter password: abc
ERROR!
Error: Invalid password. Attempts left: 2
Attempt processed
Enter username: admin
Enter password: zy
ERROR!
Error: Invalid password. Attempts left: 1
Attempt processed
Enter username: admin
Enter password: y
ERROR!
Error: Invalid password. Attempts left: 0
ERROR!
Error: Account locked due to too many failed attempts
Attempt processed
```

```java
import java.util.*;

public class ShoppingCart {

    static List<Product> catalog = new ArrayList<>();
    static List<Product> cart = new ArrayList<>();
    static double walletBalance = 5000;

    public static void main(String[] args) {
        catalog.add(new Product("Shirt", 500, 3));
        catalog.add(new Product("Shoes", 1500, 0));
        catalog.add(new Product("Watch", 2000, 5));

        Scanner sc = new Scanner(System.in);

        try {
            System.out.print("Enter product to search: ");
            String search = sc.next();
            Product p = searchProduct(search);
            System.out.println("Found: " + p.name + " Price: " + p.price + " Stock: " + p.stock);

            addToCart(p);

            System.out.print("Enter payment amount: ");
            double payAmount = sc.nextDouble();
            makePayment(payAmount);

            System.out.println("Order placed successfully. Wallet balance: " + walletBalance);

        } catch (ProductNotFoundException e) {
            System.out.println("Product Error: " + e.getMessage());
        } catch (OutOfStockException e) {
            System.out.println("Stock Error: " + e.getMessage());
```

```java
            System.out.println("Stock Error: " + e.getMessage());
        } catch (InvalidPaymentException e) {
            System.out.println("Payment Error: " + e.getMessage());
        } catch (InsufficientFundsException e) {
            System.out.println("Payment Error: " + e.getMessage());
        } catch (EmptyCartException e) {
            System.out.println("Order Error: " + e.getMessage());
        }
        sc.close();
    }

    static Product searchProduct(String name) throws ProductNotFoundException {
        for (Product p : catalog) {
            if (p.name.equalsIgnoreCase(name)) return p;
        }
        throw new ProductNotFoundException("Product not found: " + name);
    }

    static void addToCart(Product p) throws OutOfStockException {
        if (p.stock <= 0)
            throw new OutOfStockException("Product out of stock: " + p.name);
        cart.add(p);
        p.stock--;
    }

    static void removeFromCart(Product p) {
        cart.remove(p);
    }

    static void makePayment(double amount) throws InvalidPaymentException, InsufficientFundsException, EmptyCartException {
        if (cart.isEmpty())
            throw new EmptyCartException("Cart is empty, cannot proceed to payment");
        if (amount <= 0)
            throw new InvalidPaymentException("Payment amount must be positive");
        if (amount > walletBalance)
```

```
        if (amount > walletBalance)
            throw new InsufficientFundsException("Insufficient funds in wallet");
        walletBalance -= amount;
    }
}

class Product {
    String name;
    double price;
    int stock;

    Product(String name, double price, int stock) {
        this.name = name;
        this.price = price;
        this.stock = stock;
    }
}

class ApplicationException extends Exception {
    ApplicationException(String msg) { super(msg); }
}

class ProductException extends ApplicationException {
    ProductException(String msg) { super(msg); }
}

class ProductNotFoundException extends ProductException {
    ProductNotFoundException(String msg) { super(msg); }
}

class OutOfStockException extends ProductException {
    OutOfStockException(String msg) { super(msg); }
```

```
    OutOfStockException(String msg) { super(msg); }
}

class PaymentException extends ApplicationException {
    PaymentException(String msg) { super(msg); }
}

class InvalidPaymentException extends PaymentException {
    InvalidPaymentException(String msg) { super(msg); }
}

class InsufficientFundsException extends PaymentException {
    InsufficientFundsException(String msg) { super(msg); }
}

class OrderException extends ApplicationException {
    OrderException(String msg) { super(msg); }
}

class EmptyCartException extends OrderException {
    EmptyCartException(String msg) { super(msg); }
}
```

Output

```
Enter product to search: shirt
Found: Shirt Price: 500.0 Stock: 3
Enter payment amount: 700
Order placed successfully. Wallet balance: 4300.0
```

```java
class InvalidUsernameException extends Exception {
    InvalidUsernameException(String message) {
        super(message);
    }
}

class InvalidPasswordException extends Exception {
    InvalidPasswordException(String message) {
        super(message);
    }
}

class AccountLockedException extends Exception {
    AccountLockedException(String message) {
        super(message);
    }
}

public class Q6 {

    static String correctUsername = "admin";
    static String correctPassword = "1234";
    static int failedAttempts = 0;
    static final int MAX_ATTEMPTS = 3;
    static boolean locked = false;

    static void login(String username, String password)
            throws InvalidUsernameException,
                   InvalidPasswordException,
                   AccountLockedException {

        if (locked) {
            throw new AccountLockedException("Account is locked");
        }

```

```
        if (!username.equals(correctUsername)) {
            failedAttempts++;
            throw new InvalidUsernameException("Invalid username");
        }

        if (!password.equals(correctPassword)) {
            failedAttempts++;

            if (failedAttempts >= MAX_ATTEMPTS) {
                locked = true;
                throw new AccountLockedException("Account locked after maximum failed attempts");
            }

            throw new InvalidPasswordException(
                    "Invalid password. Attempts left: " + (MAX_ATTEMPTS - failedAttempts)
            );
        }

        failedAttempts = 0;
        System.out.println("Login successful");
    }

    public static void main(String[] args) {

        String[] usernames = {"admin", "admin", "admin"};
        String[] passwords = {"1111", "2222", "3333"};

        for (int i = 0; i < usernames.length; i++) {
```

```
            try {
                login(usernames[i], passwords[i]);

            } catch (InvalidUsernameException e) {
                System.out.println(e.getMessage());

            } catch (InvalidPasswordException e) {
                System.out.println(e.getMessage());

            } catch (AccountLockedException e) {
                System.out.println(e.getMessage());

            } finally {
                System.out.println("Login attempt completed");
            }
        }
    }
}
```

Output

```
ERROR!
error: can't find main(String[]) method in class: InvalidUsernameException
```